

Automating Runs with the VS API

Building and Running a VS Math Model: Overview	2
Access the Examples	2
License Manager Options.....	2
Running a VS Math Model from MATLAB.....	4
Running a VS Math Model from ANSI C.....	6
Running a VS Math Model from Visual Basic (VB)	8
Running a VS Math Model from Python	9

VS Solvers are libraries that support the building and running of VS Math Models for CarSim, TruckSim, BikeSim, and SuspensionSim. The Windows versions of these products include both 32- and 64-bit solvers, e.g., `carsim_32.dll` and `carsim_64.dll` for CarSim. For Linux, they are 64-bit `.so` library files.

On Windows, the VS Solver DLL files are normally used from the main application program in the products, e.g., `carsim.exe` or `carsim_64.exe` for CarSim. These applications, called *VS Browsers*, provide a GUI for interacting with the models and managing a database of files that are mostly read as input by the VS Math Model, or written as output by the model. Alternatively, a VS Solver may be loaded and controlled by other Windows programs.

Programs that are used mainly to load a dynamic library and use its functions are sometimes called *wrappers*.

In support of automation work in which a VS Browser is not needed, the Windows versions of the products include the wrappers `VS_SolverWrapper_CLI_32.exe` and `VS_SolverWrapper_CLI_64.exe`. These are described in the Technical Memo *VS Solver CLI Wrapper*.

There may be scenarios in which a user-defined custom wrapper is needed or preferred. This document describes examples of custom wrappers that are provided in SDK CPAR archives for use with MATLAB, C/C++, Visual Basic (VB), and Python. The remainder of this document shows how to build and run a VS Math Model from such a program.

Another Technical Memo, *Extending VS Math Models with VS Commands and the VS API*, show how VS Math Models in CarSim, TruckSim, and BikeSim are extended with custom wrappers written in MATLAB and C/C++.

The comprehensive reference manual for VS Math Models is the *VS Math Models Reference Manual*; the reference manual for VS application program interface (API) is *The VehicleSim API: Accessing and Extending VS Math Models*.

Building and Running a VS Math Model: Overview

The top-level input file for the VS Solver is a simulation control text file that in turn contains pathnames for various files of interest, including the pathname for the solver library (DLL or .so file). This file is called the *Simfile* and has the default name `simfile.sim`. Details of Simfile contents are provided the *VS Math Models Reference Manual*.

The normal way for a wrapper program to build and run a VS Math Model on Windows is:

1. Read the Simfile to get the pathname for the library file
2. Load the VS Solver library.
3. Activate API functions from the library.
4. Use the VS API function `vs_run` to build the VS Math Model and run it. This function also requires the name of the Simfile, which contains names of input and output file names that specify the layout of the model and values for parameters and tables that customize the model.
5. Unload the library.

Access the Examples

CarSim, TruckSim, and BikeSim include the examples described in this document. They are included in a CPAR archive file that is available to be added to any new database or imported into an existing database: **SDK: External Control**.

This collection of datasets is available when building a database in the Database Builder in the VS Browser (on Windows). It may also be imported into an existing database using the **File>Import Parsfile (Any Export Type)** menu item. The CPAR archives are in the folder `Resources\CPAR_Archives` in the product `_Prog` folder, e.g., `CarSim_Prog\Resources\CPAR_Archives`.

This CPAR archive include datasets needed to reproduce runs made with external wrapper programs, plus the external wrappers and the code used to build them. These are in a folder `Extensions` added to the database. It in turn has folders for building the example wrappers in different languages. For example, Figure 1 shows the contents of the folder for Custom C examples, and Figure 2 shows the contents of the `wrapper_simple` example. The code and project files were used to build two Windows programs: `wrapper_simple_Release_Win32.exe` and `wrapper_simple_Release_x64.exe`.

License Manager Options

A VS Math Model needs license information to run. When run under the control of a VS Browser on Windows, the VS Browser manages the license information automatically.

When a VS Math Model is used from software other than the VS Browser, one way to provide the necessary license information is to have the VS Browser running (it may be minimized) while you work in the environment of the external software. This method is convenient when you are making changes to vehicle datasets or test procedures from within the browser.

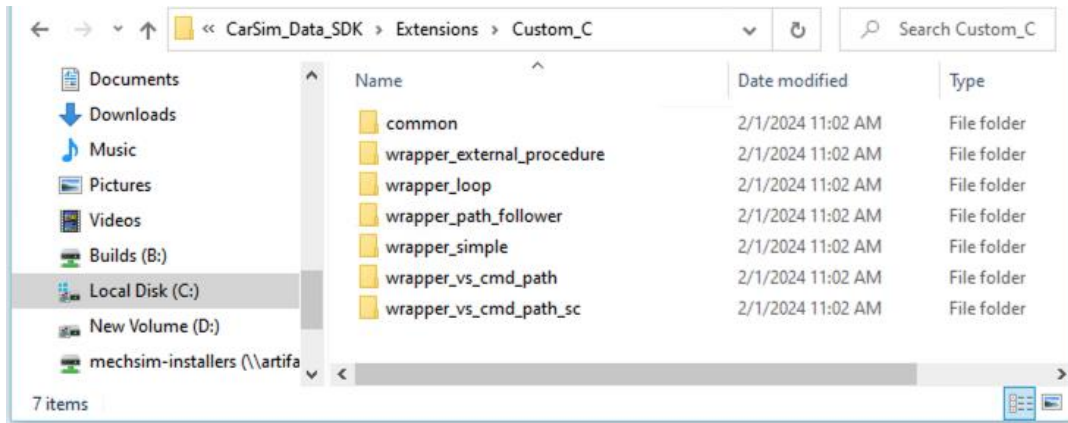


Figure 1. Folder with Custom C examples, located in the Extensions folder added with a CPAR.

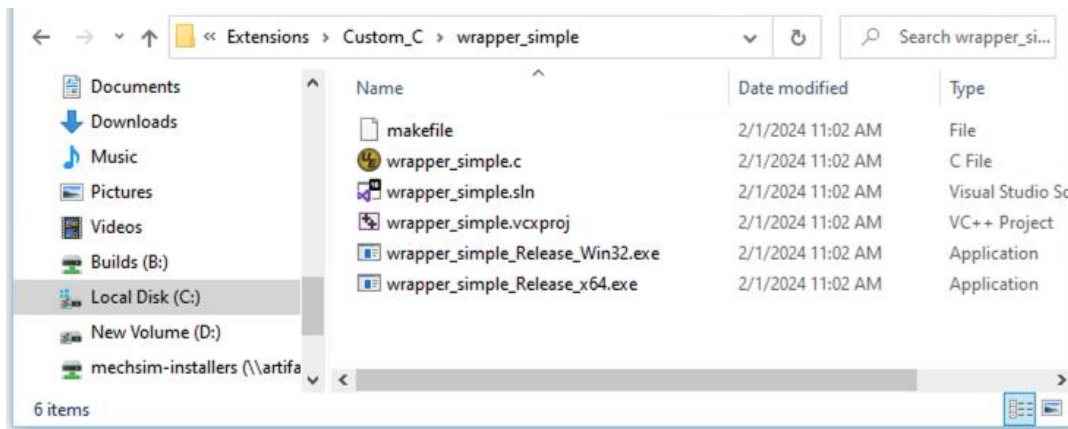


Figure 2. Source code, project files, and wrapper programs for a simple C example.

A second way to provide license information is with the stand-alone License Manager program provided with all VehicleSim products. For Windows, the program file is BSLM.exe, CSLM.exe, TSLM.exe, or SSLM.exe, located in either the _Prog\Programs folder (e.g., CarSim_Prog \Programs) or within the Utilities folder of the VS SDK. This program provides the same license control that is ordinarily provided by the VS Browser. To use it, launch the BSLM/CSLM/TSLM/SSLM .exe file (e.g., double click on the file name) before you start using any software that makes use of VS Solver DLLs. You can also make a shortcut on your desktop or in another convenient location, or add it to your start menu so it is always active after you log in. Details about the license manager are provided in a separate tech memo *The VehicleSim License Manager Utility*, covering topics such as changing the settings and running older versions of your software.

Using the License Manager application is convenient when you are using a custom interface program to produce the input files for the VS Solver, or when many runs are made in an automated environment, such as optimization. VSLM is a small program requiring minimal resources, so your system performance will not be affected by having it run in the background.

On Linux, the same license manager is used as when running with the command-line interface (cli) tool.

Running a VS Math Model from MATLAB

Building and running a VS Math Model from MATLAB requires statements in MATLAB to read a Simfile, load the VS Solver DLL, access its functions, and apply the API function `vs_run`.

If you have set up the run from the VS Browser, you typically want the Simfile to be generated automatically to use the proper datasets for the VS Math Model. To do this, make a dataset in the **Run Control** library with settings for the run (vehicle, procedure, etc.), and use the **Models** drop-down control (3) (Figure 3) to link to the library **Models: Transfer to Local Windows Directory** (4). Make a link (2) to a dataset in the linked library.

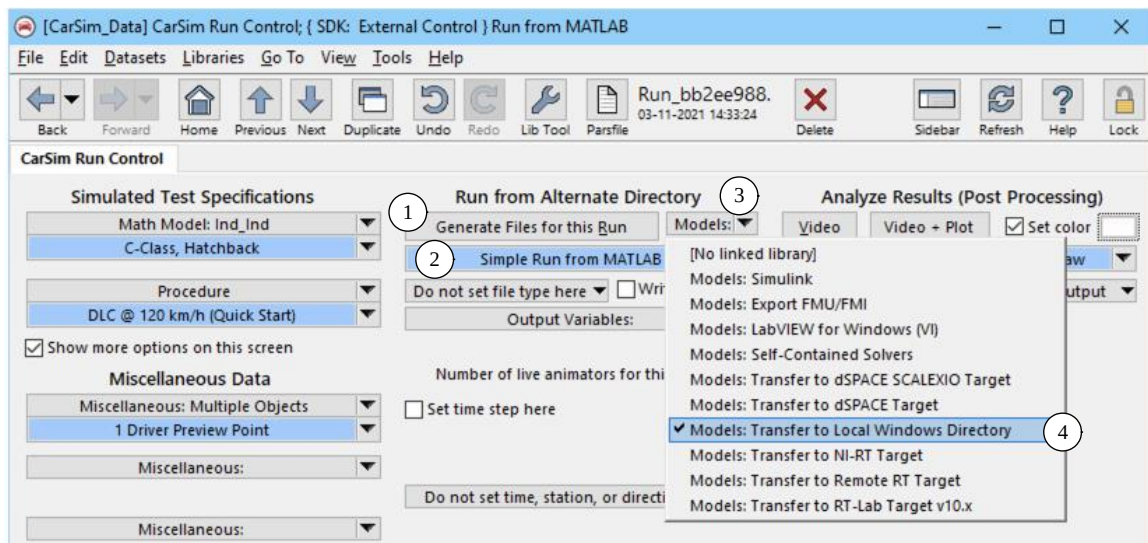


Figure 3. Link to the library **Models: Transfer to Local Windows Directory**.

Use the linked dataset to identify the folder containing the MATLAB file (1) (Figure 4). In the same dataset, use the drop-down control for choosing a 32/64 bit solver (2) to match your version of MATLAB. This will cause the pathname for the appropriate VS Solver to be written in the Simfile.

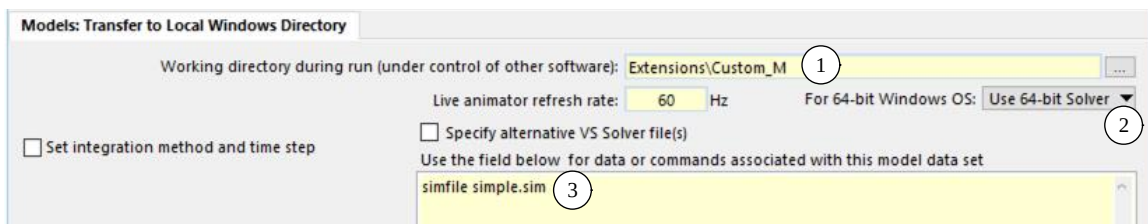


Figure 4. Specify the folder containing MATLAB files.

The dataset that specifies the directory may also be used to specify a custom Simfile name with the keyword `simfile` (3). For the simple run, the Simfile is named `simple.sim`.

Return to the **Run Control** screen (Figure 3) and click the button **Generate Files for this Run** (1) to generate the Simfile in the specified working directory, along with a matching `All.Par` file with all settings from the database that will be used by the VS Math Model for the run.

To run the simulation from MATLAB, load the M file (Figure 5). With the contents of the M file displayed (Simple.m (3)), use the **EDITOR** tab (1) to view the **Run** button (2). Click the **Run** button to execute the code in the M file, with results shown in Command Window (6). When the run is complete, plots and animations can be viewed from the VS Browser, from the Run Control screen (Figure 3).

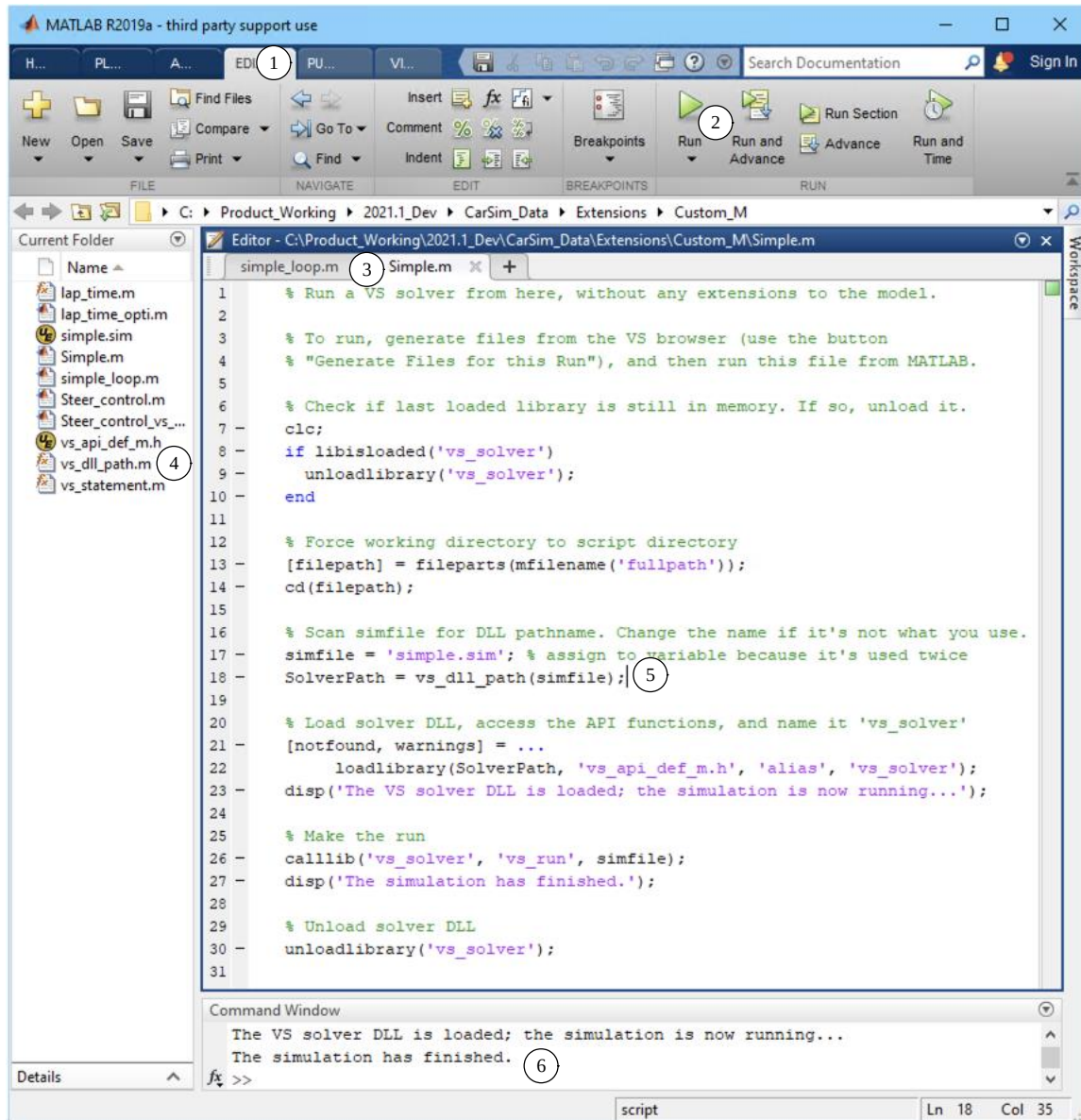


Figure 5. Loading and running a VS Solver from MATLAB.

Note The details for how you control MATLAB depend on the version.

This example performs the following steps:

1. It clears the command window and checks to ensure a DLL is not already loaded (lines 7 -10).
2. It sets the name of a variable `simfile` to the string 'simple.sim' and uses an M-file function `vs_dll_path` ((4) and (5)) to scan the specified Simfile for the location of the VS Solver DLL, and assign that pathname to the variable `SolverPath` (lines 14 and 18).
3. The MATLAB function `loadlibrary` loads the DLL along with a header file (`vs_api_def_m.h`) that gives access to VS API functions (lines 21 – 23).
4. The simulation is run with the VS API function `vs_run` and the same `simfile` variable (line 26).
5. The DLL is unloaded (line 30).

The status of the run is printed in the Command window (6).

Running a VS Math Model from ANSI C

Figure 7 (page 7) shows the source code for an ANSI C wrapper program (`solver_simple.c`) that reads a simulation control file, gets the name of the VS Solver library, and makes the run. The EXE is built from this file and another (`vs_get_api.c`) that provides access to the API functions in the VS Solver.

In the example, the name of the Simfile is an optional argument (lines 16 – 22) that defaults to "simfile.sim" if not provided (line 19). The Simfile is scanned to determine the pathname of the DLL with the function `vs_get_dll_path` (line 23). This function (from `vs_get_api.c`) looks for an explicit pathname; if not found, it is generated from the “bitness” (32 of 64) of the wrapper along with the product name (from the Simfile). When successful, the DLL is loaded (line 24). API functions are accessed with the function `vs_get_api` (line 27).

Two API functions are used in this example: `vs_run` (line 30) and `vs_get_error_message` (lines 32 and 34).

Use the **Models: Self-Contained Solvers** library to make a dataset for the wrapper (Figure 6). Check the box to use an external wrapper program (1) and specify the pathname for the compiled wrapper program (2). As noted above, the example wrapper will use the VS Solver with the same bitness as the wrapper. In this example, the wrapper name has the bitness in the name: 64 (2).

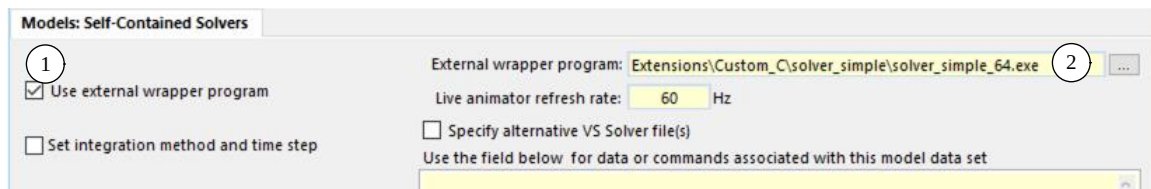


Figure 6. Identify the pathname to the compiled EXE wrapper program.

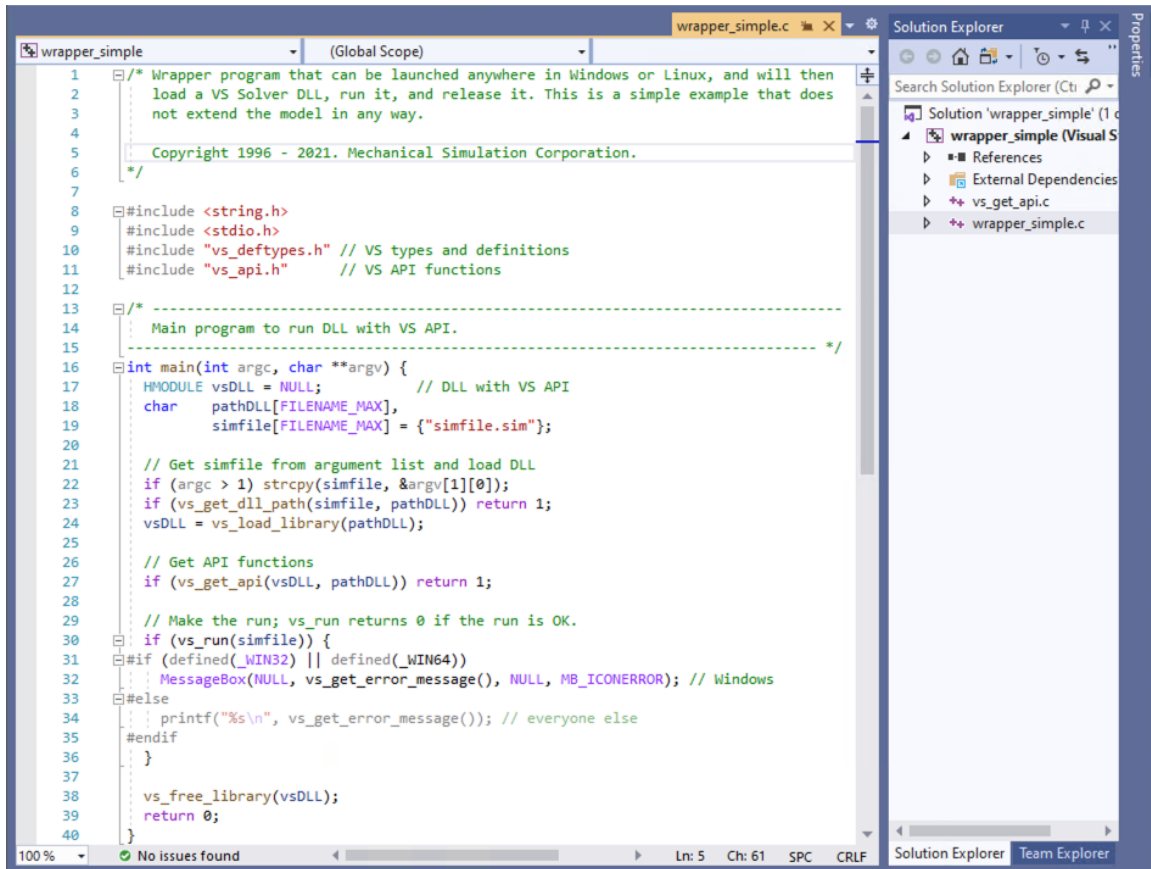


Figure 7. Loading the DLL and making a run from C.

To run a simulation with this wrapper, make a dataset in the **Run Control** screen with settings for the run (vehicle, procedure, etc.). Use the **Models** drop-down control (3) (Figure 8) to link to the library **Models: Self-Contained Solvers** (4). Make a link to the appropriate **Models** dataset in the linked library (2).

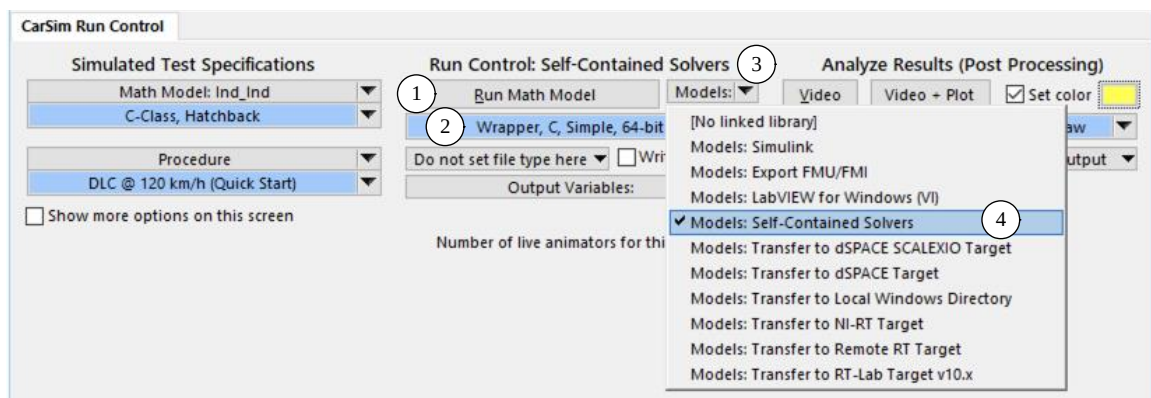


Figure 8. Link to the library Models: Self-Contained Solvers.

Run the wrapper program by clicking the **Run Math Model** button (1). The wrapper program will run as a console application (Figure 9). When operated in this manner, the behavior of the VS Math Model is the same as if the simulation were run from the Browser. Results are visualized using the

same tools, Echo files have the same appearance, and simulation results are identical to those obtained without the wrapper program.

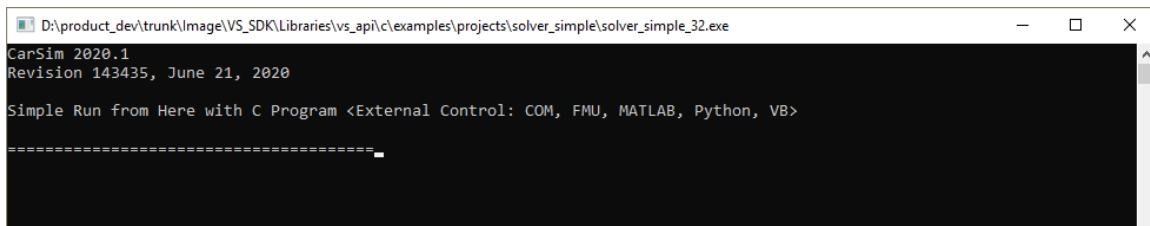


Figure 9. EXE wrapper program running as a console application.

This EXE program can be used from other environments that are not able to easily load and run DLLs directly. Because it can be given the Simfile name as an optional argument, it is not always necessary to write the Simfile in the same folder as the EXE. That is, the EXE can be deployed with a Windows run command that specifies a simulation control file with any name, in any valid Windows location. The only requirement is that the VS Browser or the VSLM utility program must be running to provide license support.

Note	As mentioned earlier, VS products include simple wrapper EXE programs that support a command-line interface:
	VS_SolverWrapper_CLI_32.exe and
	VS_SolverWrapper_CLI_32.exe. These exist to support
	automation without requiring users to create custom wrappers.

Running a VS Math Model from Visual Basic (VB)

A wrapper program can be written in Visual Basic (VB) to load the VS Solver and run a VS Math Model.

A complication is that VB uses a different internal representation for strings (e.g., the pathname of the Simfile). VehicleSim Products include a VisualBasic.NET project called VsDotNet. This project contains a Simulation class (see Simulation.vb) that provides a .NET wrapper for some of the VS API functions, such as the handling of string variables.

The example code in VS_Run.vb (Figure 10) runs a simulation using the same general method shown for MATLAB and C.

1. The name of a Simfile (the default is "simfile.sim") is assigned to a string variable pathToSimfile (lines 10 - 13).
2. An instance of the Simulation class is assigned to the variable vs (line 15), and a pathname to the DLL file is obtained and assigned to the string variable PathToVsDll by scanning the specified Simfile (line 16).
3. If a file named PathToVsDll exists (line 18), then the DLL is loaded (line 19) and some of the API functions are accessed (line 21).

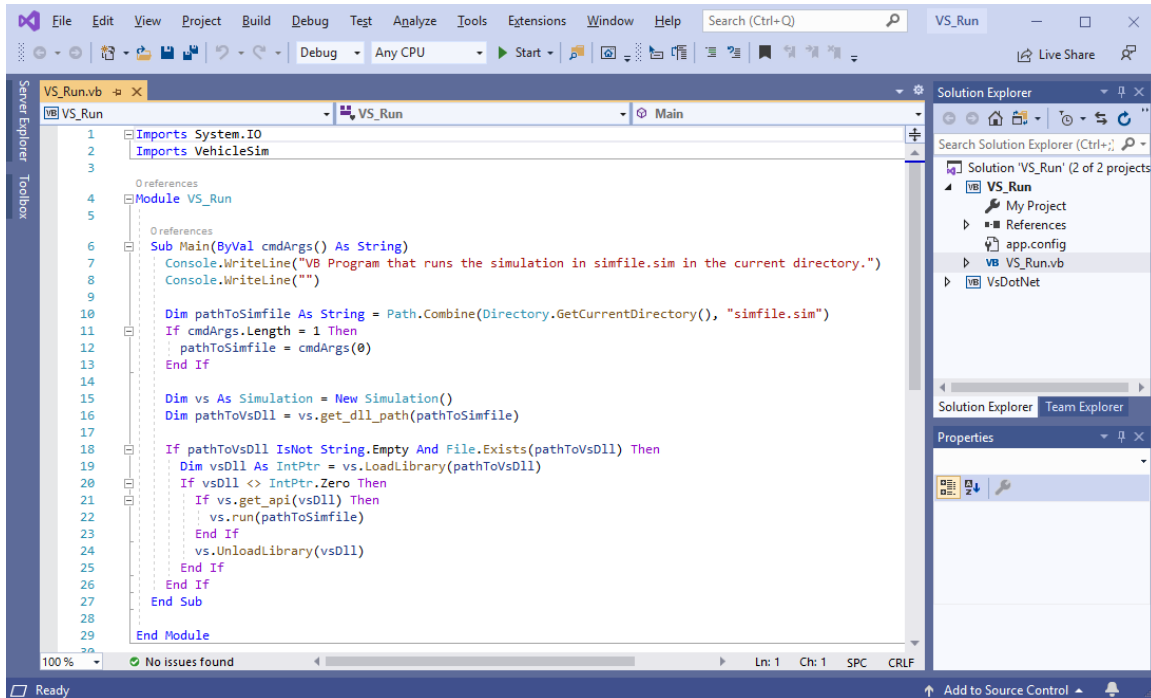


Figure 10. VB program that loads a VS Solver DLL and makes a run.

4. The API function `vs_run` is applied using the `Simulation` function `vs.run` (line 22).
5. The DLL is unloaded (line 24).

EXE programs such as this can be run from the **Run Control** screen (as shown before for the C example), or from an environment outside the VS database (as shown before for the MATLAB example).

To run from the VS Browser, link to a dataset in the **Models: Self-Contained Solvers** library (Figure 8), check the box to use an external wrapper program ⁽¹⁾ (Figure 6) and specify the pathname for the compiled wrapper program ⁽²⁾.

To run from the MSVS environment, link to a dataset from the library **Models: Transfer to Local Windows Directory** ⁽⁴⁾ (Figure 3, page 4). In the linked dataset (Figure 4, page 4), specify the directory in which the EXE file will be located ⁽¹⁾. If you are not using a default name for the Simfile, specify a name, as was done earlier for the MATLAB example ⁽³⁾.

From the **Run Control** screen (Figure 3), click the button **Generate Files for this Run** ⁽¹⁾ to generate the Simfile in the specified directory. After that, you can run the VB program from MSVS, or by double clicking on the EXE file as viewed in the Windows File Explorer.

Running a VS Math Model from Python

A wrapper program can also be written in Python to load the VS Solver and make a run.

The Python running the wrapper should match the DLLs being executed. That is, if you are referencing 32-bit DLLs, then you should run a 32-bit Python.

The example code in `VS_Run.py` (Figure 11) runs a simulation using the same general method shown for MATLAB and ANSI C.



```

16 def run_solver(args_list):
17     error_occurred = 1
18     try:
19         process_id = args_list[0]
20         path_to_sim_file = args_list[1]
21         if args_list[2] < 0:
22             num_sequential_runs = sys.maxsize
23         else:
24             num_sequential_runs = args_list[2]
25
26         solver_api = vs_solver.vs_solver()
27         system_word_size = (8 * struct.calcsize("P")) # 32 or 64
28
29         if len(args_list[3]) > 0:
30             path_to_source_vs_dll = args_list[3]
31         else:
32             path_to_source_vs_dll = solver_api.get_dll_path(path_to_sim_file)
33
34         current_os = platform.system()
35         if path_to_source_vs_dll is not None and os.path.exists(path_to_source_vs_dll):
36             if current_os == "Linux":
37                 mc_type = platform.machine()
38                 if mc_type == 'x86_64':
39                     dll_size = 64
40                 else:
41                     dll_size = 32
42             else:
43                 if " 64" in path_to_source_vs_dll:
44                     dll_size = 64
45                 else:
46                     dll_size = 32
47
48             path_to_destination_vs_dll = path_to_source_vs_dll + "_" + str(process_id)
49             shutil.copy(path_to_source_vs_dll, path_to_destination_vs_dll)
50
51             if system_word_size != dll_size:
52                 print("Python size (32/64) must match size of .dlls being loaded.")
53                 print("Python size:", system_word_size, "DLL size:", dll_size)
54             else: # systems match, we can continue
55                 vs_dll = cdll.LoadLibrary(path_to_destination_vs_dll)
56                 if vs_dll is not None:
57                     if solver_api.get_api(vs_dll):
58                         for i in range(0, num_sequential_runs):
59                             print(os.linesep + "+++++++ Starting run number: " + str(
60                                 i + 1) + " +++++++" + os.linesep)
61                             error_occurred = solver_api.run(path_to_sim_file.replace('\\\\', '\\'))
62                             if error_occurred is not 0:
63                                 print("ERROR OCCURRED: ")
64                                 solver_api.print_error()
65                                 sys.exit(error_occurred)
66                             print(os.linesep + "+++++++ Ending run number: " + str(
67                                 i + 1) + " +++++++" + os.linesep)
68             except:
69                 e = sys.exc_info()[0]
70                 print(e)
71
72     return error_occurred

```

Figure 11. Python program that loads VS Solver DLL and makes a run.

1. The name of a Simfile (the default is `simfile.sim`) and number of runs to perform (default is 1) have been passed in as arguments are assigned to variables `path_to_sim_file` and `num_sequential_runs` respectively (lines 19 - 24).

2. An instance of the `vs_solver` class is assigned to the variable `solver_api` (line 26), and a pathname to the DLL file is obtained and assigned to the string variable `path_to_vs_dll` by scanning the specified Simfile (line 32).
3. If a file named in `path_to_vs_dll` exists (line 35), then the DLL is loaded (line 55) and some of the API functions are accessed (line 57).
4. The API function `vs_run` is applied using the `vs_solver` function `solver_api.run` (line 61).

To run from the VS Browser, link to a dataset in the **Models: Self-Contained Solvers** library (Figure 8), linked dataset shown in Figure 6, and check the box to use an external wrapper program ① and specify the pathname for the wrapper program ②. The `Steer_Control.py` example, included in the VS SDK, shows an example of this usage; the VS Browser runs Python via the Windows Command Script `Steer_Control_Py.cmd`. The version of Python used (32-bit/64-bit) should match that of the used VS Solver.

To run from the external environment, link to a dataset from the library **Models: Transfer to Local Windows Directory** ④ (Figure 3, page 4). In the linked dataset (Figure 4, page 4), specify the directory in which the Python file will be located ①. If you are not using a default name for the Simfile, specify a name, as was done earlier for the MATLAB example ③.

From the **Run Control** screen (Figure 3), click the button **Generate Files for this Run** ① to generate the Simfile in the specified directory. After that, you can run the Python program from your Python IDE, or by double clicking on the .py file as viewed in the Windows File Explorer.

Note	The <code>vs_solver</code> class (as accessed with <code>vs_solver.py</code>) has been updated, as of Version 2020.1 from the old <code>Simulation</code> class. Users developing new code are encouraged to use the new <code>vs_solver</code> class, as the old class has been deprecated.
-------------	---